

CS336 Assignment 3 (scaling): Scaling Laws

Lenard Strahringer
Student ID: 06773780

Spring 2026

1 IsoFLOPs Scaling Laws (chinchilla_isoflops)

(a) Optimal Model Size

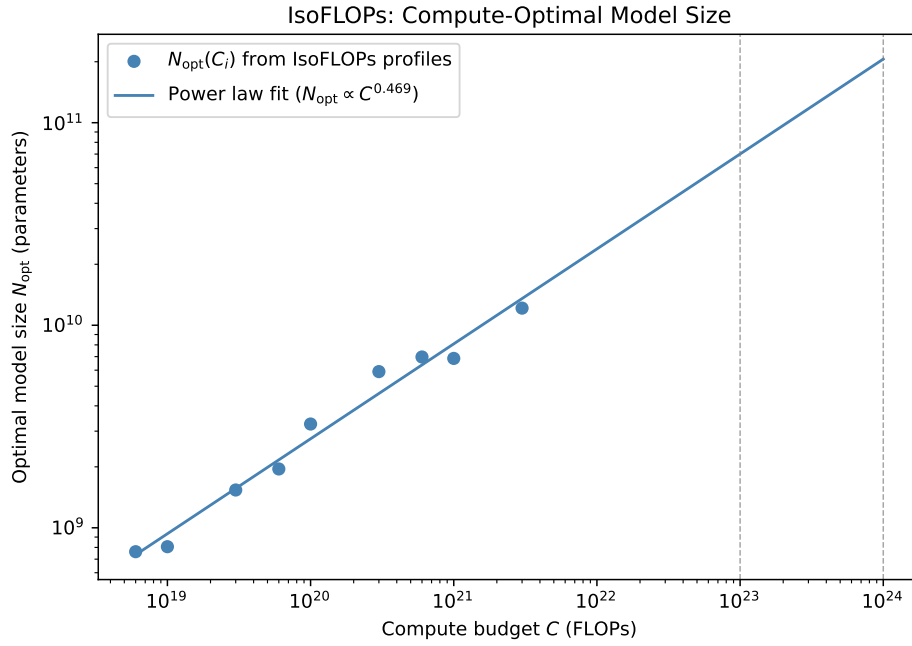


Figure 1: Best model size N_{opt} for a fixed compute budget C . Points come from IsoFLOPs profiles. The line is a power fit $N_{\text{opt}} \propto C^a$.

At $C = 10^{23}$ FLOPs the best model size is 7.0×10^{10} parameters. At $C = 10^{24}$ FLOPs it is 2.1×10^{11} parameters.

(b) Optimal Dataset Size

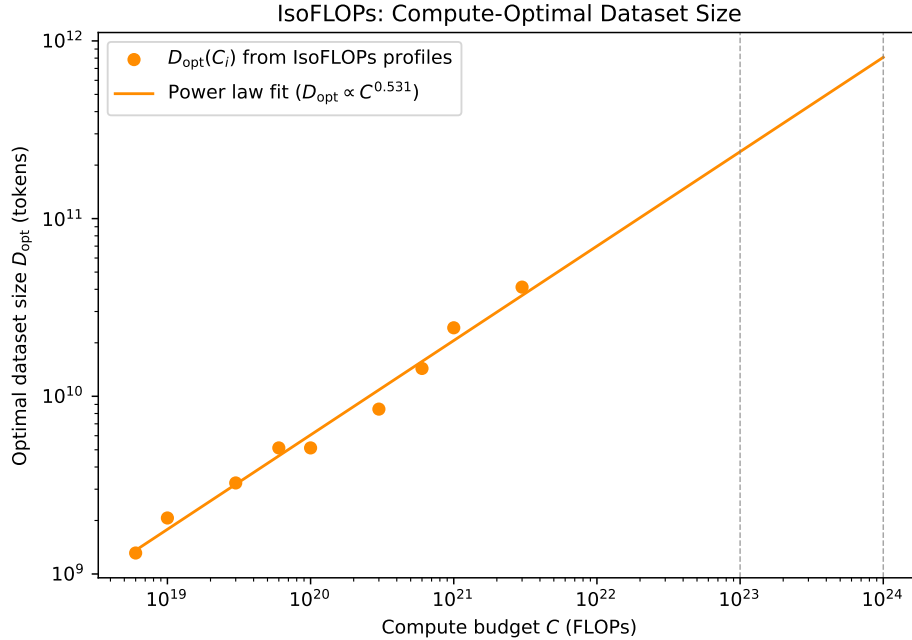


Figure 2: Best dataset size D_{opt} for a fixed compute budget C . Points come from IsoFLOPs profiles. The line is a power fit $D_{\text{opt}} \propto C^b$.

At $C = 10^{23}$ FLOPs the best dataset size is 2.4×10^{11} tokens. At $C = 10^{24}$ FLOPs it is 8.1×10^{11} tokens.

2 Constructing Scaling Laws (scaling_laws)

2.1 Experimental Design

I split a 12-hour B200 budget into three phases. The plan follows Kaplan et al. [1], Hoffmann et al. [2], and Yang et al. [3]. First I ran one short job. It checked the API, the logging path, and how much of peak FLOPs the cluster reached in practice (MFU).

Phase 1: learning rate on a small model. I need one learning rate for all later runs. I tuned it on the smallest model allowed by the API: `hidden_size = num_attention_heads · head_dim` with `head_dim = 64`. That model has $d_{\text{model}} = 128$, $L = 2$, and 1.77 M non-embedding parameters. I tried nine learning rates on a log grid from 3×10^{-4} to 3×10^{-1} . Each run used about 16 M tokens and a cosine schedule over that length (Hoffmann et al. [2], their Approach 3). I set the upper rate high on purpose so the largest value would fail. That way the best rate sits inside the grid, not on the edge.

Phase 2: how learning rate scales with width. Yang et al. [3] describe μP , where the best learning rate does not depend on width. The full recipe needs options the assignment API does not expose. I used the simpler width-scaling rule from Yang et al. [3] (Section 6) and Hoffmann et

al. [2]:

$$\eta(d) = \eta_{\text{proxy}} \cdot \left(\frac{d_{\text{proxy}}}{d} \right)^\gamma. \quad (1)$$

I treat γ as a number to fit from data. I tried three learning rates at $d_{\text{model}} \in \{320, 512\}$ around the common guess $\gamma = 1.0$. The first grids put the best rate at the low end ($\times 0.5 / \times 1 / \times 2$). I added a second round at $\times 0.25$ and $\times 0.5$ of the previous best. After that, the best rate lay inside the grid for both widths.

Phase 3: scaling-law data. I fixed η_{proxy} and γ . I trained six model families from 1.77 M to 116 M non-embedding parameters. Compute targets were $C \in \{10^{18}, 3 \times 10^{18}, 10^{19}\}$ FLOPs (Table 1). I kept depth and width near $d_{\text{model}}/L \approx 50$. Kaplan et al. [1] show that shape matters little at fixed parameter count. Two compute levels (10^{18} and 3×10^{18}) have at least four families each. That lets me compare IsoFLOPs points to the parametric fit (Hoffmann et al. [2] Approach 2).

(d_{model}, L)	N (non-embedding)	Compute levels (FLOPs)
(192, 4)	1.77 M	$\{1 \times 10^{18}\}$
(320, 6)	7.37 M	$\{1 \times 10^{18}, 3 \times 10^{18}\}$
(448, 9)	21.7 M	$\{1 \times 10^{18}, 3 \times 10^{18}, 1 \times 10^{19}\}$
(576, 10)	39.8 M	$\{1 \times 10^{18}, 3 \times 10^{18}, 1 \times 10^{19}\}$
(704, 12)	71.5 M	$\{3 \times 10^{18}, 1 \times 10^{19}\}$
(832, 14)	116.4 M	$\{1 \times 10^{18}, 3 \times 10^{18}\}$

Table 1: Phase 3 model families and per-family compute levels in my sweep.

Budget handling. The API reserves wall time up to `max_runtime_seconds` when a job starts. It returns unused time when the job ends. I set `max_runtime` to `safety_factor · C / (MFU · peak_FLOPs)` with `MFU` = 0.4 and `safety_factor` = 1.5. Values above the two-hour cap were clipped. I submitted Phase 3 in three batches (five, five, three runs). Earlier batches freed budget before the next batch started.

2.2 Phase 1 & 2 Results

The small-model sweep peaked at $\eta_{\text{proxy}} = 2.25 \times 10^{-2}$ (validation loss 5.85). The endpoints 3×10^{-4} and 3×10^{-1} were worse (7.98 and 6.68). So the grid surrounds the best value.

Phase 2 gave best rates 4.50×10^{-3} at $d_{\text{model}} = 320$ and 2.81×10^{-3} at $d_{\text{model}} = 512$. Plugging into the scaling law and averaging over the two widths,

$$\gamma = \frac{1}{2} \sum_{d \in \{320, 512\}} \frac{\log(\eta(d)/\eta_{\text{proxy}})}{\log(d_{\text{proxy}}/d)} = 1.628. \quad (2)$$

This is larger than the often-used $\gamma = 1$. Yang et al. [3] (Figure 1) show the simple scaling rule can shift the best rate by almost an order of magnitude when width grows. I use $\gamma = 1.628$ in Phase 3.

2.3 Phase 3 Outcome and Anchor Calibration

All 13 Phase 3 runs hit the time cap. From eval throughput I estimated real MFU between 1.6% and 4.7%. I had assumed 40% for planning. Three smallest jobs logged no eval block. The other ten gave one to three partial validation losses each.

I still used the partial runs. For each I set trained tokens to $D_{\text{actual}} = (\text{evals_completed}/16) \cdot D_{\text{planned}}$. I fit the law on $(N, D_{\text{actual}}, L_{\text{partial}})$. Stopped runs may bias loss upward: the schedule was cut while the learning rate was still high.

Anchor run. I checked that on one setup: $(d_{\text{model}}, L) = (576, 10)$. I trained again with `total_train_tokens` equal to D_{actual} from the truncated run. The cosine run could finish as intended (final LR $0.1 \eta_{\text{peak}}$). The two losses match within 0.001 nats:

- Truncated run (high LR throughout): $L = 3.7820$.
- Full schedule on D_{actual} : $L = 3.7813$.

The effect is tiny at this (N, D) . I fit using salvaged D_{actual} values.

2.4 Fitting the Scaling Law

I used Hoffmann et al. [2] Approach 3 (their Equations 2–3). The model is

$$\widehat{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}. \quad (3)$$

I estimated (E, A, B, α, β) with Huber loss on log-residuals ($\delta = 10^{-3}$, as in Hoffmann et al.) and L-BFGS-B. I started from eight initial guesses: $\log E \in \{\log 1, \log 2\}$, $\alpha \in \{0.3, 0.5\}$, $\beta \in \{0.3, 0.5\}$. I kept the run with the lowest objective.

On my ten salvaged points,

$$E = 3.376, \quad A = 9.66 \times 10^6, \quad B = 9.64 \times 10^6, \quad \alpha = 1.049, \quad \beta = 0.857.$$

My fitted α and β are larger than Hoffmann et al. [2]’s ($\alpha = 0.34$, $\beta = 0.28$). The ratio $\alpha/\beta = 1.22$ is almost the same as Chinchilla’s 1.21. The best split between N and D at fixed compute depends only on that ratio (Hoffmann et al. [2] Equation 5). So my allocation prediction matches theirs in that sense. The larger exponents likely come from my narrow N range ($5.4\times$ here vs. $230\times$ in Hoffmann et al.).

2.5 Quality of Fit

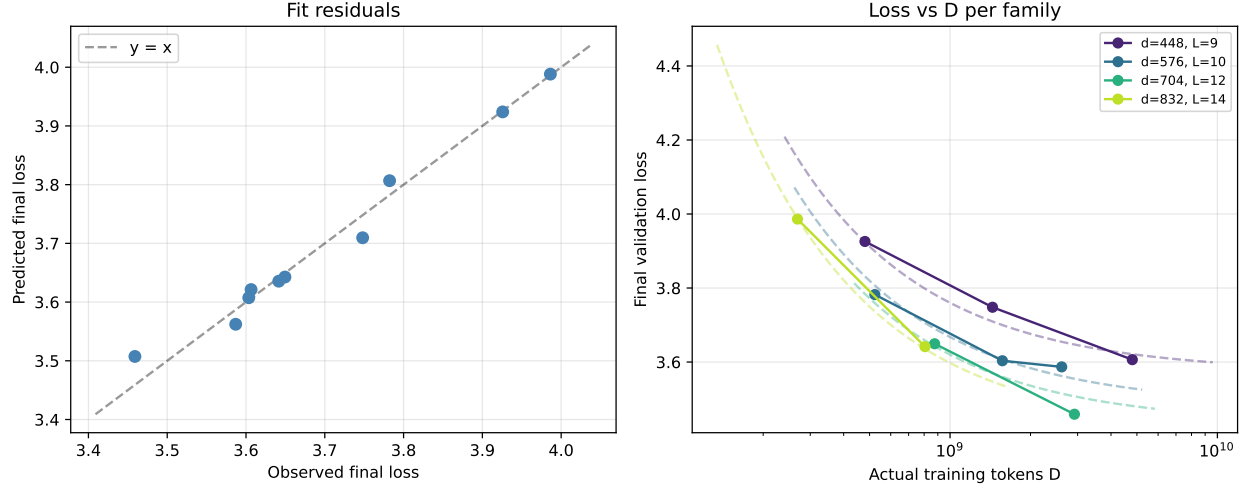


Figure 3: Left: predicted vs. observed loss for my ten salvaged Phase 3 runs. Right: loss vs. tokens by model family; dashed lines are the fit at fixed N .

All ten residuals are small ($\max |r| = 0.048$ nats, RMSE 0.025 nats); see Figure 3. Two limits apply outside this range:

- N only spans 21.7M to 116M ($5.4\times$). Hoffmann et al. [2] stress that exponents depend strongly on the range.
- Token counts give $D/N \in [12, 220]$. Most points are above the Chinchilla rule $D/N \approx 20$. Extrapolation to other D/N may be weak.

2.6 Predicted Optimal Configuration for 48 B200-Hours

Hoffmann et al. [2] Equation 5 with unconstrained compute gives $N^* = 1.23 \times 10^9$ and $D^* = 1.05 \times 10^{11}$ for 7.78×10^{20} FLOPs (48 B200-hours at peak). At measured MFU $\approx 5\%$ on my runs, that job would need about 211 wall-clock hours. It does not fit the 48-hour limit.

I minimized $\hat{L}(N, D)$ under $6ND = M \cdot \text{peak_FLOPs} \cdot 48 \text{ h}$. I set $M = 0.05$ from my 116M-parameter runs. Larger models often get higher MFU on this hardware. I have no direct measure. I also reduced D by 20% vs. the constrained optimum to keep time slack (≈ 9.6 hours at $M = 0.05$).

Table 2 lists the result. Predicted validation loss:

$$L^* = \hat{L}(3.13 \times 10^8, 1.66 \times 10^{10}) = 3.4051. \quad (4)$$

2.7 Hyperparameter Choices

I follow Kaplan et al. [1] for depth and width. I follow Hoffmann et al. [2] for the schedule and optimizer: cosine to total tokens, $10\times$ LR decay, AdamW with $\beta_2 = 0.95$. Peak learning rate follows Yang et al. [3] via the width rule with my $\gamma = 1.628$. That yields $\eta = 6.90 \times 10^{-4}$ from $\eta_{\text{proxy}} = 2.25 \times 10^{-2}$ at $d_{\text{proxy}} = 128$.

Hyperparameter	Value
<code>num_hidden_layers</code>	22
<code>hidden_size</code>	1088
<code>num_attention_heads</code>	17
<code>head_dim</code>	64
<code>intermediate_size</code>	3072
<code>total_train_tokens</code>	1.66×10^{10}
<code>train_batch_size</code>	128
Peak learning rate η	6.90×10^{-4}
Warmup fraction	0.05
Final-LR fraction	0.10 (cosine to 0.1η)
AdamW (β_1, β_2)	(0.9, 0.95)
Weight decay	0.01
Gradient clip norm	1.0

Table 2: Suggested hyperparameters for my 48 B200-hour run. Non-embedding parameters $N = 3.13 \times 10^8$. The model satisfies `hidden_size` = `num_attention_heads` · `head_dim` and `num_key_value_heads` = `num_attention_heads`.

References

- [1] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models.” 2020.
- [2] J. Hoffmann *et al.*, “Training Compute-Optimal Large Language Models.” 2022.
- [3] G. Yang *et al.*, “Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.” 2022.